Document number:	P0033R1
Date:	2015-10-24
Project:	Programming Language C++, Library Evolution Working Group
Reply-to:	Jonathan Wakely <cxx@kayari.org>, Peter Dimov <pdimov@pdimov.com>

Re-enabling `shared_from_this` (revision 1)

I. Table of Contents

II. Revision History

- Added "unambiguous" and "implicitly convertible" requirements to "*enables `shared_from_this`*" definition.
- Added caveat about data races.
- Added `noexcept` to `weak_from_this`.
- Recommend a single feature-testing macro, rather than two.

III. Introduction

The class template `enable_shared_from_this` is very weakly specified, making it hard to reason about its behaviour in some situations and risking implementation divergence. This proposal provides a more precise specification and where the desired behaviour is unclear recommends standardising the behaviour of `boost::enable_shared_from_this`. In addition the `weak_from_this` member functions of the Boost version are proposed for inclusion in C++.

IV. Motivation and Scope

LWG issue 2529 shows the following program which has unspecified behaviour in C++14:

```
#include <memory>

using namespace std;

int main()
{
  struct X : public enable_shared_from_this<X> { };
  auto xraw = new X;
  shared_ptr<X> xp1(xraw);  // #1
  {
    shared_ptr<X> xp2(xraw, [](void*) { });  // #2
  }
  xraw->shared_from_this();  // #3
}
```

The standard says that #1 should set `xraw->__weak_this` to share ownership with `xp1`. It is unclear whether #2 should update it to share ownership with `xp2` instead. At #3 all the preconditions for calling `shared_from_this()` are met:

> *Requires:* `enable_shared_from_this<T>` shall be an accessible base class of `T`. `*this` shall be a subobject of an object `t` of type `T`. There shall be at least one `shared_ptr` instance `p` that owns `&t`.

But depending on what happened at #2 it might be impossible to meet the postconditions:

> *Returns:* A `shared_ptr<T>` object `r` that shares ownership with `p`.
> *Postconditions:* `r.get() == this`.

If `xraw->__weak_this` was updated at #2 then it is impossible to meet the postcondition because the `weak_ptr` is expired. For the case where #2 does not update the `weak_ptr` a different example can be constructed that again makes it impossible to meet the postcondition, by ensuring that whichever `shared_ptr` shares ownership with the `weak_ptr` is made to release its ownership before the call to `shared_from_this()`.

The root of the problem is that `enable_shared_from_this` is underspecified, leaving the precise semantics up to implementors. The aim of this proposal is to clearly specify the effects so that the behaviour of the example above is predictable and portable.

The proposed specification is **is a breaking change** for the three major standard library implementations tested, however it is believed to be unlikely that any working code is relying on the current behaviour.

In addition to the improved specification for `enable_shared_from_this` two new member functions are proposed. The `weak_from_this` functions are directly analogous to the existing `shared_from_this` functions but they return a `weak_ptr` instead. These new functions are a pure extension and have been provided in Boost since the recent 1.58 release. It is possible to obtain a `weak_ptr` today via `weak_ptr<T>(shared_from_this())` but the proposed new functions achieve the same thing more efficiently (with fewer reference count modifications), they more clearly express the intent, and they can be used in additional situations.

The proposed wording removes the preconditions on `shared_from_this` so that it is now well-defined to call it on an object which is not owned by any `shared_ptr`, in which case `shared_from_this` would throw an exception. `weak_from_this().lock()` is a non-throwing alternative to `shared_from_this()` that returns an empty `shared_ptr` when the object is not owned by any `shared_ptr`. This can be used in situations where the overhead of an exception is undesirable and in environments that disable exceptions entirely.

`weak_from_this` can also be used in the object's destructor, after the `weak_this` member has expired. `weak_from_this` can be used to get a `weak_ptr` that still shares ownership with any other `weak_ptr` objects that still exist after the last "strong" reference has been released by the last `shared_ptr`.

Finally, the proposed changes would also resolve LWG issue 2179. The improved specification clarifies that the behaviour of the example in the issue is undefined, as there is no sharing between the two instances.

V. Design Decisions

The important design decision is whether or not a `weak_ptr` member that already shares ownership with one `shared_ptr` should be modified by subsequent constructions of unique `shared_ptr` objects.

Existing practice is consistent across the Dinkumware, GNU and LLVM implementations of `std::shared_ptr`, with all three updating the `weak_ptr` member on subsequent constructions, however it is not clear whether this is by design or simply an oversight due to not considering the possibility (which is the case for the GNU implementation). On the other hand, `boost::shared_ptr` does not update the `weak_ptr` member and this is a deliberate choice that was made in response to user feedback[1] and consideration of the alternatives. When asked about the above example Peter Dimov said:

> "Based on our experience with boost::enable_shared_from_this, there are real-world cases where you very much want the above to work, and there are no cases in which you want it to not work."
>
> "The first owner is generally what one wants returned from shared_from_this. There is one corner case here and it occurs when the object acquires a second owner when the first owner is already dead. The Boost implementation does replace the owner in this case.

In support of the argument that the first owner is the one that should be associated with the `weak_this` member, Peter also points out that the code doing:

```
{
  shared_ptr<X> xp2(xraw, [](void*) { });  // #2
}
```

may occur somewhere deep inside a library that only gets a raw pointer, and that therefore it's much better if it doesn't break `xp1->shared_from_this()`, because the owner of `xp1` would not expect it.

The experience from Boost is persuasive, so the proposed change is to standardise the Boost behaviour, even though that requires all three standard library implementations to change their previous behaviour.

VI. Technical Specification

> For the purposes of SG10, I recommend a feature-testing macro named `__cpp_lib_enable_shared_from_this` to indicate conformance to the new specification, including the new `weak_from_this` members.

Add a new paragraph before paragraph 1 of [util.smartptr.shared.const]:

> In the constructor definitions below, *enables `shared_from_this` with `p`*, for a pointer `p` of type `Y*`, means that if `Y` has an unambiguous and accessible base class that is a specialization of `enable_shared_from_this` ([util.smartptr.enab]), then `remove_cv_t<Y>*` shall be implicitly convertible to `T*` and the constructor evaluates the statement:
>
> ```
> if (p != nullptr && p->weak_this.expired())
>   p->weak_this = shared_ptr<remove_cv_t<Y>>(*this, const_cast<remove_cv_t<Y>*>(p));
> ```
>
> The assignment to the `weak_this` member is not atomic and conflicts with any potentially concurrent access to the same object (1.10).

Edit paragraph 4:

> *Effects:* Constructs a `shared_ptr` object that *owns* the pointer `p`. Enables `shared_from_this` with `p`.

Edit paragraph 9:

> *Effects:* Constructs a `shared_ptr` object that *owns* the object `p` and the deleter `d`. The first and second constructors enable `shared_from_this` with `p`. The second and fourth constructors shall use a copy of `a` to allocate memory for internal use.

Edit paragraph 29:

> *Effects:* Equivalent to `shared_ptr(r.release(), r.get_deleter())` when `D` is not a reference type, otherwise `shared_ptr(r.release(), ref(r.get_deleter()))`. Enables `shared_from_this` with the value that was returned by `r.release()`.

Add two member functions and an exposition-only member to the class synopsis in [util.smartptr.enab]:

```
namespace std {
  template<class T> class enable_shared_from_this {
  protected:
    constexpr enable_shared_from_this() noexcept;
    enable_shared_from_this(enable_shared_from_this const&) noexcept;
    enable_shared_from_this& operator=(enable_shared_from_this const&) noexcept;
    ~enable_shared_from_this();
  public:
    shared_ptr<T> shared_from_this();
    shared_ptr<T const> shared_from_this() const;
    weak_ptr<T> weak_from_this() noexcept;
    weak_ptr<T const> weak_from_this() const noexcept;
  private:
    mutable weak_ptr<T> weak_this; // exposition only
  };
} // namespace std
```

Edit paragraph 4:

> *Effects:* ~~Constructs an `enable_shared_from_this<T>` object.~~Value-initializes `weak_this`.

Add a note after paragraph 5:

> *Returns:* `*this`.
>
> [*Note:* `weak_this` is not changed. — *end note*]

Remove the redundant destructor specification and paragraph 6:

> ~~`~enable_shared_from_this`~~
>
> ~~*Effects:* Destroys `*this`.~~

Replace paragraphs 7, 8 and 9:

```
shared_ptr<T>       shared_from_this();
shared_ptr<T const> shared_from_this() const;
```

> ~~*Requires:* `enable_shared_from_this<T>` shall be an accessible base class of `T`. `*this` shall be a subobject of an object `t` of type `T`. There shall be at least one `shared_ptr` instance `p` that *owns* `&t`.~~
> ~~*Returns:* A `shared_ptr<T>` object `r` that *shares ownership with* `p`.~~

Returns: `shared_ptr<T>(weak_this).`

Add definitions for the new members after the replaced paragraph 9:

```
weak_ptr<T>       weak_from_this() noexcept;
weak_ptr<T const> weak_from_this() const noexcept;
```

Returns: `weak_this`.

Remove the note in paragraphs 10 and 11:

~~[*Note:* A possible implementation is shown below:~~

```
template<class T> class enable_shared_from_this {
private:
  weak_ptr<T> __weak_this;
protected:
  constexpr enable_shared_from_this() : __weak_this() { }
  enable_shared_from_this(enable_shared_from_this const &) { }
  enable_shared_from_this& operator=(enable_shared_from_this const &) { return *this; }
  ~enable_shared_from_this() { }
public:
  shared_ptr<T> shared_from_this() { return shared_ptr<T>(__weak_this); }
  shared_ptr<T const> shared_from_this() const { return shared_ptr<T const>(__weak_this); }
};
```

~~The `shared_ptr` constructors that create unique pointers can detect the presence of an `enable_shared_from_this` base and assign the newly created `shared_ptr` to its `__weak_this` member. — *end note*]~~

VII. Implementation Experience

Boost's `boost::enable_shared_from_this` has done the `expired()` check for years, and has provided `weak_from_this` since the Boost 1.58.0 release. The libstdc++ development sources were recently altered to do the expired check.

VIII. Acknowledgements

Thanks to Howard Hinnant for his comments on the issue.

IX. References

[1] Ticket #2584: boost::enable_shared_from_this + boost.python library, Boost Trac, Nicolas Lelong, 2008-12-12.

[2] [boost] enable_shared_from_this::weak_from_this() request, Boost mailing list, Marat Abrarov, 2011-06-11.